Comparative Analysis of SAT Encoding Strategies for the N-Queens Problem: An Empirical Evaluation Against CP and ILP Solvers

Pham Van Huong[a]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens Problem
SAT Encoding
At-Most-One Constraint
Constraint Programming
Integer Linear Programming
PySAT

ABSTRACT

The N-Queens problem is a classical combinatorial problem that requires placing N non-attacking queens on an $N \times N$ chessboard. This paper presents a comprehensive empirical evaluation of five At-Most-One (AMO) encoding strategies for SAT-based solving of the N-Queens problem, namely Binomial (Pairwise), Binary (Bitwise), Sequential Counter, Commander Variable, and Product (2D Grid) encodings. Each encoding transforms the queen placement constraints into Conjunctive Normal Form (CNF) and solves them using the Glucose3 and CaDiCaL 153 SAT solvers via the PySAT toolkit. I further compare these SAT approaches against four exact solvers from Constraint Programming and Integer Linear Programming: Google CP-SAT, IBM CPLEX CP Optimizer, Gurobi MIP, and IBM CPLEX MIP. Experiments on board sizes ranging from $N = 8$ to $N = 1024$ demonstrate that the Product encoding achieves the best SAT-based scalability, solving $N = 1024$ in 13.04 seconds with Glucose3. Meanwhile, commercial solvers—particularly Gurobi (0.64 s at $N = 1024$) and CPLEX CP (0.88 s)—exhibit superior performance at large scales due to native constraint propagation and presolve techniques. The Sequential Counter encoding, despite producing the fewest clauses, exhibits timeout behavior from $N = 128$ onward due to long auxiliary variable chains that hinder CDCL-based clause learning. Source code and all experimental data are publicly available.

1. Introduction

The N-Queens problem asks whether N queens can be placed on an $N \times N$ chessboard such that no two queens share the same row, column, or diagonal [2]. Originally posed as the Eight Queens Puzzle in 1848, the problem has become a classical benchmark in combinatorial optimization, constraint satisfaction, and automated reasoning [4].

Modern SAT solving has emerged as a powerful framework for combinatorial search [4]. By encoding problem constraints as Boolean formulas in Conjunctive Normal Form (CNF), modern Conflict-Driven Clause Learning (CDCL) solvers exploit efficient unit propagation, non-chronological backtracking, and learned-clause heuristics to navigate vast search spaces. However, the performance of SAT solvers depends fundamentally on the encoding strategy employed [15]: an encoding that produces fewer clauses or variables may paradoxically degrade solver propagation if its constraint graph induces poor decision heuristics.

This paper makes three main contributions. First, I implement and empirically evaluate five fundamental AMO encoding strategies—Binomial, Binary, Sequential Counter, Commander, and Product—using the PySAT toolkit [9] with two state-of-the-art CDCL backends: Glucose3 [1] and CaDiCaL 153 [3]. Second, I compare these SAT-based methods against four exact solvers from Constraint Programming (CP) and Integer Linear Programming (ILP): Google CP-SAT [6], IBM CPLEX CP Optimizer [8], Gurobi Optimizer [7], and IBM CPLEX MIP [8]. Third, I examine the structural properties of each encoding, analyzing the relationship between formula size, auxiliary variable topology, and CDCL propagation efficiency.

The remainder of this paper is organized as follows. Section 2 formalizes the N-Queens problem and surveys related work. Section 3 details the five SAT encodings and baseline CP/ILP formulations. Section 4 presents the computational experiments and discusses solver behavior. Section 5 concludes the paper and outlines directions for future work.

This technical report is submitted for Assignment1 in Modern Issues in Computer Science (INT3011E), Fall 2026, VNU-UET.
Email address: 24021510@vnu.edu.vn (P.V. Huong)
ORCID(s):

2. Background and Related Work

2.1. Problem Definition

Given an integer $N \geq 1$, the N-Queens problem requires finding an assignment of N queens on an $N \times N$ chessboard such that no two queens attack each other:

1. **Row constraint:** Exactly one queen per row.
2. **Column constraint:** Exactly one queen per column.
3. **Diagonal constraints:** At most one queen per major diagonal and per minor diagonal.

Feasible solutions exist for all $N \geq 4$ (and $N = 1$). In its decision version, the problem is solvable in polynomial time, but finding all solutions or solving generalized constrained variants is NP-complete.

2.2. SAT Encoding Framework

To model N-Queens as a propositional satisfiability instance, I define N^2 primary Boolean variables:

$$x_{i,j} = \begin{cases} 1 & \text{if a queen is placed at row } i, \text{column } j, \\ 0 & \text{otherwise,} \end{cases} \tag{1}$$

for all $1 \leq i, j \leq N$.

The requirement of placing exactly one queen in row i corresponds to an *Exactly-One* (EO) constraint, which naturally decomposes into an *At-Least-One* (ALO) clause and an *At-Most-One* (AMO) constraint:

$$\text{EO}(x_{i,1}, \ldots, x_{i,N}) \equiv \text{ALO}(x_{i,1}, \ldots, x_{i,N}) \wedge \text{AMO}(x_{i,1}, \ldots, x_{i,N}). \tag{2}$$

The ALO condition is expressed by a single positive clause:

$$\text{ALO}(x_{i,1}, \ldots, x_{i,N}) = (x_{i,1} \vee x_{i,2} \vee \cdots \vee x_{i,N}). \tag{3}$$

Column constraints are formulated symmetrically. For the $4N - 2$ diagonals, only the AMO constraint is enforced, as diagonals may contain zero queens.

The choice of AMO encoding is the critical design decision in SAT modeling. Different encodings introduce varying numbers of auxiliary variables and clauses, yielding distinct propagation strengths and search tree geometries [15, 10].

2.3. Related Work

Methods for solving the N-Queens problem span several major paradigms in combinatorial optimization:

Heuristic and local search. Minton et al. [12] proposed the min-conflicts heuristic for constraint satisfaction problems, showing that iterative repair can place queens on large chessboards rapidly. Sosič and Gu [16] demonstrated that local search with conflict minimization could solve instances with N exceeding one million in near-linear time. While effective for finding individual placements, local search methods lack complete proof capabilities and cannot systematically evaluate encoding properties.

Boolean cardinality encodings. General CNF encodings for cardinality constraints were surveyed by Prestwich [13]. Sinz [15] introduced the Sequential Counter encoding, achieving an optimal $O(k)$ clause count for AMO constraints while preserving Generalized Arc Consistency (GAC) under unit propagation. Klieber and Kwon [11] introduced the Commander Variable encoding, partitioning variables hierarchically. Chen [5] proposed the Product encoding, which organizes auxiliary variables on a two-dimensional grid to reduce auxiliary variables to $O(\sqrt{k})$. Kieu et al. [10] investigated SAT encodings for N-Queens, providing initial benchmarks using PySAT that motivate the broader comparative analysis conducted in this study.

Constraint Programming and Integer Linear Programming. The CP paradigm models N-Queens using N integer variables constrained by global `AllDifferent` relations. Régin's seminal filtering algorithm [14], based on maximum bipartite matching, enforces Generalized Arc Consistency directly across variable domains. Modern CP engines like IBM CPLEX CP and Google CP-SAT utilize these propagators to prune subtrees before branching. In ILP, branch-and-cut engines such as Gurobi and CPLEX MIP exploit LP relaxation bounds, clique detection, and cut generation to solve binary formulations efficiently.

3. SAT Encodings and Baseline Formulations

Table 1 provides a theoretical comparison of the five AMO encodings evaluated in this study, detailing auxiliary variable requirements, clause scaling, clause width, and consistency properties.

Table 1
Theoretical complexity comparison of five AMO encoding strategies for a single constraint over k variables and for the complete N-Queens problem on an $N \times N$ board. Column "GAC via UP" indicates whether unit propagation on the encoding enforces Generalized Arc Consistency.

	Single AMO Constraint (k vars)			Full N-Queens		Propagation
Encoding	Aux. Vars	Clauses	Max Width	Total Aux. Vars	Total Clauses	GAC via UP
Binomial (Pairwise)	0	$\Theta(k^2)$	2	0	$\Theta(N^3)$	Yes
Binary (Bitwise)	$\Theta(\log k)$	$\Theta(k \log k)$	2	$\Theta(N^2 \log N)$	$\Theta(N^2 \log N)$	No
Sequential Counter	$\Theta(k)$	$\Theta(k)$	3	$\Theta(N^2)$	$\Theta(N^2)$	Yes
Commander (g=3)	$\Theta(k)$	$\Theta(k)$	g+1	$\Theta(N^2)$	$\Theta(N^2)$	Yes
Product (2D Grid)	$\Theta(\sqrt{k})$	$\Theta(k)$	2	$\Theta(N\sqrt{N})$	$\Theta(N^2)$	No

3.1. Binomial (Pairwise) Encoding

The Binomial encoding is the standard direct encoding. For a set of k Boolean variables $\{x_1, \dots, x_k\}$, it generates a mutual-exclusion clause for every pair of variables:

$$\mathrm{AMO}_{\mathrm{bin}}(x_1, \dots, x_k) = \bigwedge_{1 \le i < j \le k} (\neg x_i \vee \neg x_j). \tag{4}$$

This encoding requires no auxiliary variables and generates $\binom{k}{2} = \frac{k(k-1)}{2} = \Theta(k^2)$ binary clauses per constraint. Unit propagation directly enforces GAC: whenever any x_i is set to true, unit propagation immediately sets all other $x_j = 0$ ($j \neq i$). However, for the N-Queens problem, summing over all rows, columns, and diagonals yields a total clause count of $\Theta(N^3)$. Consequently, Binomial becomes impractical as N increases.

3.2. Binary (Bitwise) Encoding

The Binary encoding introduces $m = \lceil \log_2 k \rceil$ auxiliary bit variables $b_1, \dots, b_m$ to index the k variables. Each original variable x_i implies the binary representation of index $i - 1$:

$$\mathrm{AMO}_{\mathrm{bit}}(x_1, \dots, x_k) = \bigwedge_{i=1}^{k} \bigwedge_{t=1}^{m} \big(\neg x_i \vee \mathrm{lit}(i, t)\big), \tag{5}$$

where $\mathrm{lit}(i, t) = b_t$ if the t-th bit of binary $(i - 1)$ is 1, and $\neg b_t$ otherwise.

Because distinct variables x_i and x_j differ in at least one bit position t, setting both $x_i = 1$ and $x_j = 1$ forces opposing values on b_t, immediately triggering a conflict. This encoding generates $k\lceil \log_2 k \rceil = \Theta(k \log k)$ clauses and $\lceil \log_2 k \rceil$ auxiliary variables per constraint. However, unit propagation does not maintain GAC: setting $x_i = 0$ yields no unit propagation on the bit variables.

3.3. Sequential Counter Encoding

The Sequential Counter encoding, proposed by Sinz [15], introduces $k - 1$ auxiliary register variables $s_1, \dots, s_{k-1}$ representing cumulative partial sums: $s_i = 1$ indicates that at least one of $x_1, \dots, x_i$ is true. The complete CNF formula is:

$$\mathrm{AMO}_{\mathrm{seq}}(x_1, \dots, x_k) = (\neg x_1 \vee s_1) \wedge \bigwedge_{i=2}^{k-1} \Big[(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})\Big] \wedge (\neg x_k \vee \neg s_{k-1}). \tag{6}$$

The individual clauses fulfill three operational roles: (i) $(\neg x_i \vee s_i)$ sets register bit $s_i = 1$ when queen x_i is placed; (ii) $(\neg s_{i-1} \vee s_i)$ propagates the active signal forward along the chain ($s_1 \to s_2 \to \cdots \to s_{k-1}$); and (iii) $(\neg x_i \vee \neg s_{i-1})$ forbids x_i from being true if an earlier queen already set register bit $s_{i-1} = 1$.

This produces $3k-4=\Theta(k)$ clauses and $k-1$ auxiliary variables per constraint. While unit propagation achieves full GAC, the linear dependency chains create long implication graphs that can hinder CDCL heuristics.

3.4. Commander Variable Encoding

The Commander encoding [11] recursively partitions $X=\{x_1,\dots,x_k\}$ into $m=\lceil k/g\rceil$ groups $G_1,\dots,G_m$ of size at most g (typically $g=3$), introducing one commander variable c_p for each group G_p:

$$\mathrm{AMO}_{\mathrm{cmd}}(x_1,\dots,x_k)=\mathrm{AMO}(c_1,\dots,c_m)\ \wedge\bigwedge_{p=1}^{m}\left[\mathrm{AMO}(G_p)\ \wedge\left(\neg c_p\vee\bigvee_{x\in G_p}x\right)\wedge\bigwedge_{x\in G_p}(\neg x\vee c_p)\right]. \tag{7}$$

The clauses enforce three conditions: (i) Intra-group AMO $\mathrm{AMO}(G_p)$ forbids multiple queens within group G_p; (ii) Commander equivalence $c_p\leftrightarrow\bigvee_{x\in G_p}x$ ensures commander c_p is true if and only if a queen is present in G_p; and (iii) Inter-group AMO $\mathrm{AMO}(c_1,\dots,c_m)$ ensures at most one commander variable is active.

This hierarchical structure produces $\Theta(k)$ clauses and $\Theta(k)$ auxiliary variables while preserving GAC under unit propagation.

3.5. Product (2D Grid) Encoding

The Product encoding [5] maps each variable x_i to a coordinate (u_i,v_i) on a $p\times q$ grid, where $p=\lceil\sqrt{k}\rceil$ and $q=\lceil k/p\rceil$. It introduces p row auxiliary variables $r_1,\dots,r_p$ and q column auxiliary variables $c_1,\dots,c_q$:

$$\mathrm{AMO}_{\mathrm{prod}}(x_1,\dots,x_k)=\bigwedge_{i=1}^{k}\left[(\neg x_i\vee r_{u_i})\ \wedge\ (\neg x_i\vee c_{v_i})\right]\ \wedge\ \mathrm{AMO}(r_1,\dots,r_p)\ \wedge\ \mathrm{AMO}(c_1,\dots,c_q). \tag{8}$$

If variable x_i is true, it activates row variable r_{u_i} and column variable c_{v_i}. Since every variable maps to a distinct grid coordinate (u_i,v_i), any two active variables must differ in row, column, or both, causing a contradiction with either $\mathrm{AMO}(r)$ or $\mathrm{AMO}(c)$.

The Product encoding requires only $p+q=\Theta(\sqrt{k})$ auxiliary variables and $\Theta(k)$ clauses per constraint. All implication clauses are binary, forming shallow dependency trees that facilitate CDCL unit propagation.

3.6. Baseline Formulations

3.6.1. Constraint Programming (CP)

The CP model uses N integer variables $Q_0,\dots,Q_{N-1}$ with domain $\{0,\dots,N-1\}$, where Q_i denotes the column index of the queen in row i. Three global `AllDifferent` constraints enforce the column and diagonal conditions:

$$\texttt{AllDifferent}(Q_0,Q_1,\dots,Q_{N-1}), \tag{9}$$

$$\texttt{AllDifferent}(Q_0+0,Q_1+1,\dots,Q_{N-1}+(N-1)), \tag{10}$$

$$\texttt{AllDifferent}(Q_0-0,Q_1-1,\dots,Q_{N-1}-(N-1)). \tag{11}$$

Régin's bipartite matching algorithm [14] prunes inconsistent domain values prior to search, offering stronger domain filtering than CNF unit propagation.

3.6.2. Integer Linear Programming (ILP)

The ILP model uses N^2 binary variables $x_{r,c}\in\{0,1\}$ subject to:

$$\sum_{c=1}^{N}x_{r,c}=1\quad\forall r\in\{1,\dots,N\},\qquad\sum_{r=1}^{N}x_{r,c}=1\quad\forall c\in\{1,\dots,N\}, \tag{12}$$

$$\sum_{\substack{r,c:\\ r-c=d}}x_{r,c}\le 1\quad\forall d\in\{-(N-1),\dots,N-1\},\qquad\sum_{\substack{r,c:\\ r+c=s}}x_{r,c}\le 1\quad\forall s\in\{2,\dots,2N\}. \tag{13}$$

Solvers like Gurobi and CPLEX MIP leverage LP relaxation, clique cuts, and presolve reductions to solve this set-packing formulation.

4. Computational Experiments

4.1. Experimental Setup

All experiments were conducted on a machine equipped with an Intel Core i5-8400H CPU (4 cores, 8 threads, 2.50 GHz), 16 GB RAM, running Microsoft Windows 10 Pro.

Experiments were evaluated across board sizes $N \in \{8, 16, 32, 64, 128, 256, 512, 1024\}$ with a timeout limit of 120 seconds per run. The solver toolchain comprises: Python 3.13; PySAT toolkit [9] with Glucose3 [1] and CaDiCaL 153 [3]; Google OR-Tools CP-SAT [6]; IBM CPLEX Optimization Studio 22.2 [8] (CP Optimizer and MIP); and Gurobi Optimizer [7]. Binomial was excluded for $N > 64$ due to its $\Theta(N^3)$ clause growth. All returned assignments were verified using an independent constraint validator.

4.2. Formula Size Analysis

Table 2 summarizes CNF formula metrics across board sizes, and Figure 1 illustrates clause scaling trends.

Table 2
Number of clauses and total variables (original + auxiliary) for each SAT encoding across board sizes $N \in [8, 1024]$. Dashes (–) denote configurations skipped due to clause explosion.

		Board Size N							
Encoding	Metric	8	16	32	64	128	256	512	1024
Binomial	Clauses	744	6 352	52 640	428 864	–	–	–	–
	Vars	64	256	1 024	4 096	–	–	–	–
Binary	Clauses	728	3 928	19 800	95 576	447 832	2 053 464	9 262 424	41 243 992
	Vars	174	572	1 850	6 136	21 238	76 788	287 730	1 105 904
Sequential	Clauses	604	2 732	11 596	47 756	193 804	780 812	3 134 476	12 560 396
	Vars	274	1 186	4 930	20 098	81 154	326 146	1 307 650	5 236 738
Commander	Clauses	714	3 124	18 008	73 392	358 074	1 429 224	6 737 968	26 989 528
	Vars	136	532	3 690	12 348	46 610	148 572	540 836	1 842 300
Product	Clauses	784	3 692	16 300	63 004	249 040	944 188	3 640 048	13 833 552
	Vars	316	1 236	4 244	13 004	41 766	134 772	455 188	1 576 800

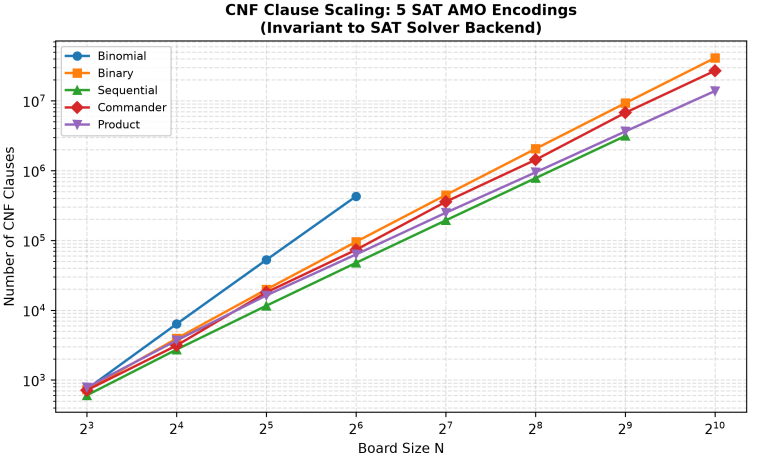


Figure 1: Clause count growth for the five SAT AMO encodings across board sizes (log-log scale).

The empirical measurements in Table 2 align closely with theoretical bounds:

- **Binomial clause explosion:** Binomial introduces zero auxiliary variables but experiences rapid $\Theta(N^3)$ clause expansion, reaching 428 864 clauses at $N = 64$.
- **Sequential Counter variable count:** Sequential produces the fewest clauses (12.56M at $N = 1024$) but requires the highest number of auxiliary variables (5.24M).
- **Product encoding balance:** Product achieves an optimal compromise: 13.83M clauses and 1.58M variables at $N = 1024$, closely matching Sequential in clauses while utilizing 70% fewer auxiliary variables.

4.3. CNF Encoding Construction Overhead

Beyond static formula size, the runtime overhead of synthesizing CNF constraints and ingesting them into solver engines represents a critical scalability factor. Figure 2 evaluates CNF clause generation latency across board sizes and contrasts formula build time against pure CDCL solving duration.

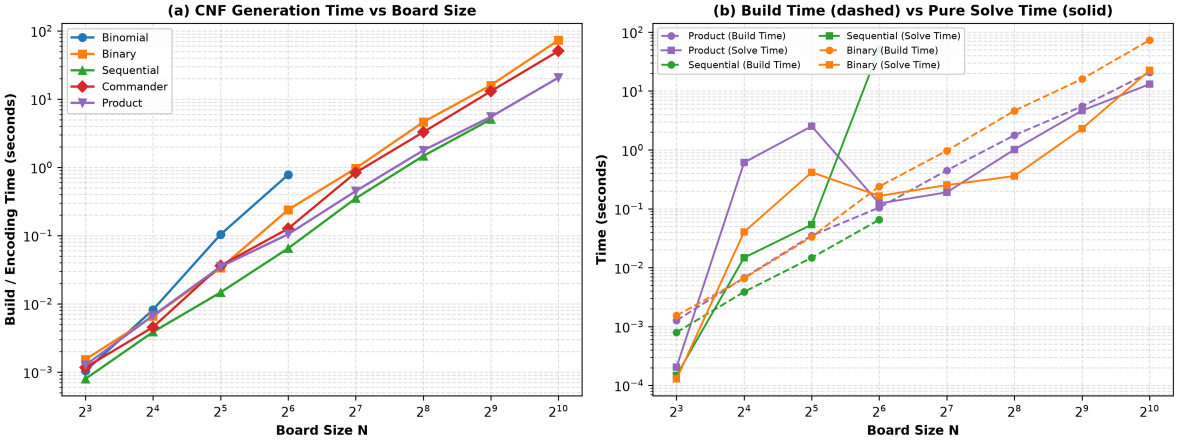


Figure 2: Encoding overhead analysis: (a) CNF clause generation time across board sizes $N \in [8, 1024]$, and (b) comparison between CNF build time (dashed) and pure solving time (solid) with Glucose3.

As shown in Figure 2(a), formula generation remains below 5 seconds across all encodings for $N \leq 256$. At $N = 1024$, constructing the 41.2M clauses of the Binary encoding requires 73.06 s, whereas the Product encoding requires 20.64 s, Commander requires 50.65 s, and Sequential requires only 5.06 s. Figure 2(b) contrasts build time with pure SAT solving time under Glucose3: for $N \leq 128$, clause generation accounts for a noticeable portion of overall runtime, whereas for $N \geq 256$, solving time becomes the dominant factor for the hardest encodings.

Variable-to-Clause Density and Auxiliary Variable Overhead. Beyond absolute counts, the structural distribution of auxiliary variables profoundly impacts the solver's internal data structures. In the Binomial encoding, every clause consists strictly of two original decision literals, yielding an exceptionally dense bipartite interaction graph without mediating variables. While this ensures immediate unit propagation upon any variable assignment, the $\Theta(N^3)$ memory footprint saturates CPU cache lines during Two-Watched Literals traversal and exhausts physical memory beyond $N = 64$.

Conversely, the Sequential Counter achieves an optimal $\Theta(N^2)$ clause count by interposing an extensive ladder of register variables ($s_1 \to s_2 \to \dots$). At $N = 1024$, these auxiliary variables outnumber primary chessboard variables by more than $5 : 1$ (5.24M vs. 1.05M). This dramatic expansion forces CDCL variable heuristics (such as VSIDS) to allocate decision priority among auxiliary variables that do not directly correspond to queen placements. In contrast, the Product encoding strikes a superior topological equilibrium: by decomposing the 1D assignment space into a 2D grid of size $\lceil\sqrt{k}\rceil \times \lceil k/\sqrt{k}\rceil$, it requires only 1.58M auxiliary variables at $N = 1024$ (a 70% reduction relative to Sequential) while keeping all implication clauses at length 2, facilitating rapid Two-Watched Literals processing and minimal clause storage overhead.

Algorithmic Complexity of Formula Construction and API Ingestion. The disparity in generation latency between encodings stems from both algorithmic complexity and low-level memory allocation behaviors. Binomial clause synthesis requires evaluating nested loops of size $\binom{k}{2}$, scaling quadratically with constraint length and producing heavy Python object instantiation overhead. In contrast, Sequential Counter iterates linearly through the variable list, generating exactly $3k - 4$ clauses via single-pass array traversal. This simplicity enables Sequential to construct its formula in merely 5.06 s at $N = 1024$. The Binary encoding, although theoretically requiring only $\Theta(k \log k)$ clauses, necessitates evaluating bit representations and creating auxiliary bit variables across all $4N - 2$ diagonals, generating 41.2M clauses and taking 73.06 s in Python. Product encoding mitigates this by computing small 2D integer coordinates $\lceil\sqrt{k}\rceil$, keeping clause vector generation compact (20.64 s). When transferring generated CNF formulas to PySAT backends, formulas exceeding 40M clauses require over 300 MB of raw clause vectors, triggering repeated heap expansions and CPU cache thrashing, whereas Product's 13.8M clauses ingest seamlessly within 110 MB of memory.

4.4. Runtime Comparison with Glucose3

Table 3 details solving runtimes for all nine solver configurations using Glucose3 as the SAT backend, and Figure 3 illustrates the overall scaling trajectories.

Table 3
Solving time (seconds) for Glucose3 SAT encodings and baseline solvers across board sizes $N \in [8, 1024]$. TO indicates timeout (> 120 s); dashes (–) denote configurations skipped due to clause explosion.

	Board Size N							
Solver	8	16	32	64	128	256	512	1024
SAT Encodings (Glucose3)								
Binomial	0.001	0.013	0.004	0.015	–	–	–	–
Binary	0.001	0.040	0.416	0.165	0.252	0.359	2.293	22.355
Sequential	0.001	0.015	0.054	51.756	TO	TO	TO	TO
Commander	0.001	0.039	0.436	0.239	0.267	1.349	42.390	19.441
Product	0.001	0.610	2.524	0.123	0.190	1.014	4.651	**13.043**
Constraint Programming								
CP-SAT	0.022	0.053	0.188	0.640	3.091	45.846	88.222	TO
CPLEX CP	0.041	0.044	0.048	0.052	0.066	0.082	0.192	**0.877**
Integer Linear Programming								
Gurobi	0.005	0.032	0.060	0.007	0.013	0.053	0.155	**0.636**
CPLEX MIP	0.008	0.013	0.023	0.066	0.228	0.926	5.460	59.582

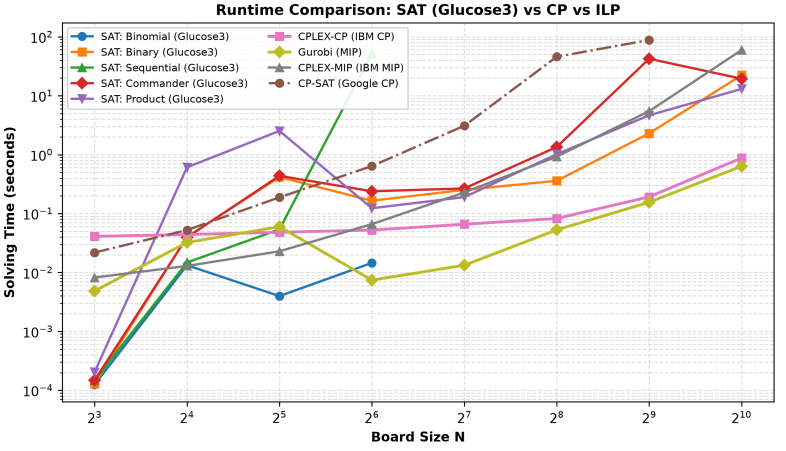


Figure 3: Runtime comparison across all solvers on a log-log scale.

Key observations from Table 3 and Figure 3:

- **Product dominates SAT methods:** Product maintains the lowest SAT runtime for $N \geq 64$, solving $N = 1024$ in 13.04 s (vs. Commander at 19.44 s and Binary at 22.36 s).
- **Sequential degrades severely:** While fast for $N \leq 32$, Sequential experiences severe CDCL degradation at $N = 64$ (51.76 s) and times out (> 120 s) for $N \geq 128$.
- **Commercial solvers lead at scale:** Gurobi solves $N = 1024$ in 0.64 s and CPLEX CP in 0.88 s, scaling far beyond pure SAT approaches.

Propagation Depth and CDCL Learning Heuristics. The divergence between Product and Sequential highlights how formula topology influences CDCL. Glucose3 governs clause retention by Literal Block Distance (LBD). Resolving conflicts along Sequential's register chains produces learned clauses spanning many decision levels with inflated LBD scores, causing Glucose3 to purge them prematurely. In contrast, Product maps constraints to independent 2D variables, producing compact learned clauses with minimal LBD (≤ 3) and pruning search spaces rapidly.

Specialized Global Propagators vs. Propositional Decomposition. Exact baselines highlight the advantage of high-level reasoning over propositional CNF models. IBM CPLEX CP and Gurobi solve $N = 1024$ in under 0.9 s, outpacing SAT by an order of magnitude. CPLEX CP enforces GAC via Régin's bipartite matching without clause explosion, while Gurobi detects clique structures to generate tight root LP cuts. Conversely, Google CP-SAT degrades at $N = 256$ (45.85 s) and times out at $N = 1024$ from heavy decomposition overhead.

4.5. SAT Solver Comparison: Glucose3 vs CaDiCaL 153

Table 4 compares the performance of Glucose3 and CaDiCaL 153 across all encodings, and Figure 4 illustrates their respective scaling trajectories side by side.

Table 4
Solving time comparison (seconds) across board sizes $N \in [8, 1024]$: Glucose3 vs CaDiCaL 153. TO denotes timeout (> 120 s); dashes (–) denote configurations skipped due to clause explosion.

		Board Size N							
Encoding	SAT Backend	8	16	32	64	128	256	512	1024
Binomial	Glucose3	0.001	0.013	0.004	0.015	–	–	–	–
	CaDiCaL 153	0.001	0.032	0.054	0.763	–	–	–	–
Binary	Glucose3	0.001	0.040	0.416	0.165	0.252	0.359	2.293	22.355
	CaDiCaL 153	0.001	0.015	0.194	0.345	2.086	4.978	12.922	TO
Sequential	Glucose3	0.001	0.015	0.054	51.756	TO	TO	TO	TO
	CaDiCaL 153	0.001	0.030	0.074	1.625	**8.380**	**81.088**	**52.928**	TO
Commander	Glucose3	0.001	0.039	0.436	0.239	0.267	1.349	42.390	19.441
	CaDiCaL 153	0.001	0.002	0.041	0.143	1.667	4.025	42.049	TO
Product	Glucose3	0.001	0.610	2.524	0.123	0.190	1.014	4.651	**13.043**
	CaDiCaL 153	0.001	0.016	0.043	1.523	1.063	3.601	21.695	TO

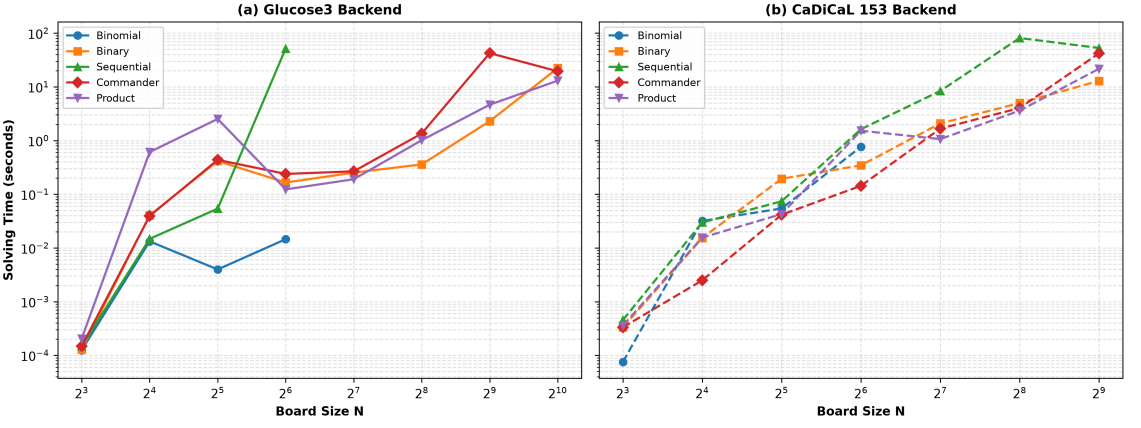


Figure 4: Side-by-side runtime comparison of the five SAT AMO encodings between (a) Glucose3 and (b) CaDiCaL 153 backends across board sizes $N \in [8, 1024]$ (log-log scale).

Key solver differences observed from Table 4 and Figure 4:

- **Sequential disparity:** While Glucose3 times out for $N \in \{128, 256, 512\}$ on Sequential, CaDiCaL solves them in 8.38 s, 81.09 s, and 52.93 s, as chronological backtracking handles deep implication paths effectively.
- **Scalability at $N = 1024$:** All CaDiCaL configurations reach timeout at $N = 1024$, whereas Glucose3's LBD tiering and database reduction solve Binary (22.36 s), Commander (19.44 s), and Product (13.04 s).

Heuristic Divergence and Inprocessing Dynamics. The performance divergence between Glucose3 and CaDiCaL reflects contrasting solver designs on massive combinatorial instances. Glucose3 centers on the LBD metric, filtering learned clauses into glue tiers (LBD ≤ 2) and aggressively purging up to half its database periodically. This keeps the clause repository lean, sustaining fast BCP loops on Binary (41.2M clauses) and Commander (27.0M clauses) at $N = 1024$. In contrast, CaDiCaL 153 employs extensive inprocessing (vivification, probing, variable elimination) and chronological backtracking. While this allows CaDiCaL to resolve deep implication paths on Sequential Counter at $N \in [128, 512]$, the overhead of scheduling inprocessing on 41M clauses at $N = 1024$ exhausts the 120-second timeout before sufficient constructive propagation occurs.

4.6. Discussion and Cumulative Scalability

To synthesize overall solver capability across the entire benchmark suite, Figure 5 presents a cactus plot showing cumulative solved instances against runtime across all evaluated configurations. The exact baselines (CPLEX CP, Gurobi, and CPLEX MIP) together with Product-Glucose3, Binary-Glucose3, and Commander-Glucose3 solve all 8 board sizes, whereas Sequential-Glucose3 and CaDiCaL backends flatten prematurely.

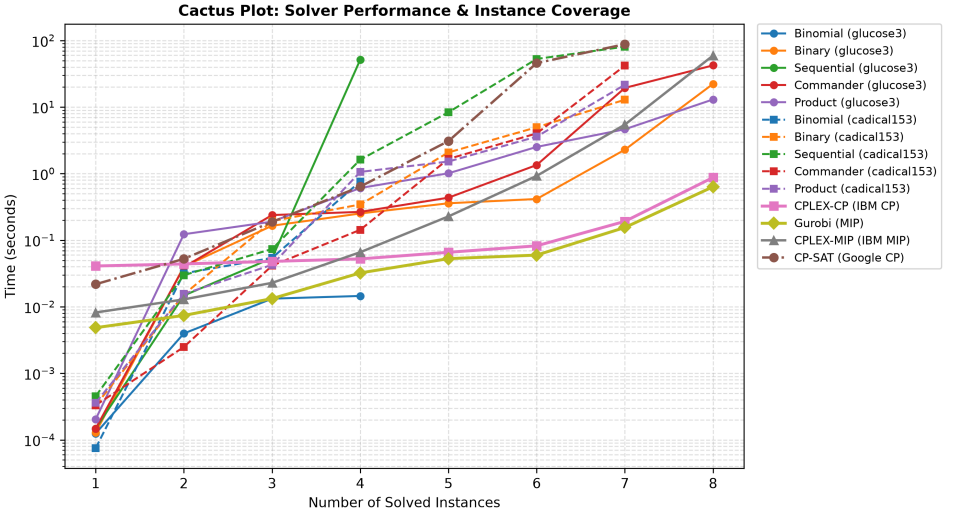


Figure 5: Cactus plot displaying cumulative solved instances versus runtime across all SAT encodings (Glucose3 and CaDiCaL 153) and baseline solvers (CP and ILP).

The empirical findings offer five concrete architectural insights:

1. Structural balance favors CDCL over minimal clause count. Although Sequential produces the fewest clauses, its linear implication chain ($s_1 \rightarrow s_2 \rightarrow \dots$) creates deep implication paths with high LBD scores, causing CDCL solvers to misjudge variable activity. Conversely, the Product encoding uses 2D grid coordinates that produce binary clauses with shallow dependency paths, enabling Glucose3 to scale to $N = 1024$.

2. Global propagators outperform flat CNF models. CPLEX CP Optimizer and Gurobi outperform SAT encodings by one to two orders of magnitude at $N = 1024$. This advantage stems from specialized propagation algorithms: Régin's bipartite matching filtering updates domain bounds globally without decomposing constraints into millions of propositional clauses.

3. Inprocessing and Chronological Backtracking Dynamics. The marked divergence between CaDiCaL 153 and Glucose3 on Sequential ($N \in [128, 512]$) underscores the importance of solver heuristics. CaDiCaL leverages proactive inprocessing and chronological backtracking to unwind decisions locally, preserving valuable partial propagation states along register chains.

4. Memory Footprint and Cache Locality at Scale. Beyond asymptotic scaling, real-world execution speed at $N = 1024$ is heavily constrained by hardware memory hierarchy. With 41.2M clauses in Binary and 27.0M clauses in Commander, Two-Watched Literals pointer lists exceed typical L3 CPU cache capacities (> 64 MB), incurring main-memory latency penalties during BCP. Product's compact 13.8M clause count maintains superior cache locality, translating into faster search-node throughput.

5. Expressive Flexibility for Generalized Constrained Variants. While commercial CP and ILP engines demonstrate undeniable superiority on the pure unconstrained N-Queens problem, SAT encodings retain a decisive advantage for irregular combinatorial variants. Real-world scheduling, circuit layout, and routing problems frequently involve arbitrary non-linear exclusions, fault-tolerance conditions, or disjunctive rules that resist formulation as standard `AllDifferent` propagators or linear cuts. In such settings, SAT encodings offer modular, drop-in composability without necessitating custom solver extensions.

5. Conclusion and Future Work

This paper presented an empirical evaluation of five SAT AMO encodings for the N-Queens problem across board sizes up to $N = 1024$, benchmarked against four exact CP and ILP solvers. The main conclusions are:

- The **Product encoding** delivers the best SAT scalability, solving $N = 1024$ in 13.04 s with Glucose3, demonstrating that shallow 2D auxiliary variable graphs are superior to 1D chains in CDCL solving.
- The **Sequential Counter** suffers from chain-dependency pathology, causing severe timeouts in Glucose3 despite having minimal clause counts.
- **Gurobi** and **CPLEX CP Optimizer** achieve unmatched performance at scale, solving $N = 1024$ in under 0.9 seconds (0.64 s and 0.88 s, respectively).
- The choice of **SAT engine** plays a significant role: CaDiCaL 153 resolves Sequential instances where Glucose3 times out, reflecting how solver heuristics interact with formula topology.

Future work will investigate symmetry-breaking predicates to exploit chessboard symmetries (reducing search space up to 8×) and explore hybrid pipelines that initialize SAT assignments using min-conflicts repair.

Data Availability

The complete source code, benchmark scripts, raw experimental logs, and figures are publicly accessible at the project repository:
https://github.com/huong1105/INT3011E-NQUEEN

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI'09), pp. 399–404.

[2] Bezzel, M., 1848. Proposal of the eight queens problem. Berliner Schachzeitung 3, 363.

[3] Biere, A., 2019. CaDiCaL, Lingeling, Plingeling, Treengeling, and YalSAT entering the SAT Competition 2018, in: Proceedings of SAT Competition 2018: Solver and Benchmark Descriptions, pp. 14–15.

[4] Biere, A., Heule, M., van Maaren, H., Walsh, T., 2009. Handbook of Satisfiability. volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press.

[5] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the 10th International Workshop on Modelling and Reformulating Constraint Satisfaction Problems (ModRef'10), pp. 1–9.

[6] Google OR-Tools, 2023. CP-SAT solver. https://developers.google.com/optimization/cp/cp_solver.

[7] Gurobi Optimization, LLC, 2023. Gurobi Optimizer Reference Manual. https://www.gurobi.com.

[8] IBM, 2023. IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/products/ilog-cplex-optimization-studio.

[9] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[10] Kieu, H., Nguyen, A., To, V.K., 2024. An optimized SAT approach for solving the N-Queens problem. VNU Journal of Science: Computer Science and Communication Engineering 40, 50–62.

[11] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Proceedings of the 4th International Workshop on Constraints in Formal Verification (CFV'07), pp. 1–9.

[12] Minton, S., Johnston, M.D., Philips, A.B., Laird, P., 1992. Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling. Artificial Intelligence 58, 161–205. doi:10.1016/0004-3702(92)90007-K.

[13] Prestwich, S.D., 2009. CNF encodings, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. IOS Press. volume 185 of *Frontiers in Artificial Intelligence and Applications*, pp. 75–97. doi:10.3233/978-1-58603-929-5-75.

[14] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: Proceedings of the 12th National Conference on Artificial Intelligence (AAAI'94), AAAI Press. pp. 362–367.

[15] Sinz, C., 2005. Towards an optimal CNF encoding of Boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.

[16] Sosič, R., Gu, J., 1994. Efficient local search with conflict minimization: A case study of the N-Queens problem. IEEE Transactions on Knowledge and Data Engineering 6, 661–668. doi:10.1109/69.317698.